

48. Déplacer un lutin

Une position, quatre boutons — et surtout pas de front.

1. Ce qu'il faut comprendre

Ici l'on veut que ça bouge **tant que le doigt reste appuyé**. Pas de front, donc : le front sert à ce qui doit arriver une fois, pas à ce qui doit durer.

Savoir lequel des deux on veut est la moitié du travail sur cette machine.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **312 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	47 o	← la console, à l'allumage – la mise en route de la console
main()	93 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	16 o	← CopierTuiles – 0 lettre sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

sprite() × 1	23 o sur place, dans main()
bouton() × 4	20 o sur place, dans main() + LireManette (37 o)
image() × 1	6 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : déplace un personnage avec les 4 flèches, un pixel par image.  
//  
// x et y sont deux variables : la place du lutin en pixels.  
// À chaque image, on regarde les flèches, on change x ou y,  
// puis on repose le lutin à sa nouvelle place.  
  
#include <Tuile>    // un dessin de 8 × 8 pixels  
#include <bouton>    // lit un bouton de la manette  
#include <sprite>    // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — HEROS

```
Tuile HEROS = {  
    "..###..", ".#-##-#.", "#####", "#.####.#",  
    "#####", "..#..#..", ".#....#.", "##....##",  
};
```



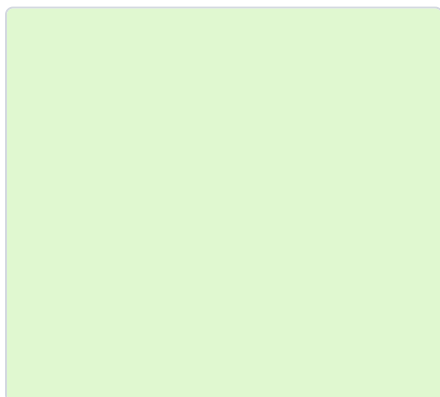
à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

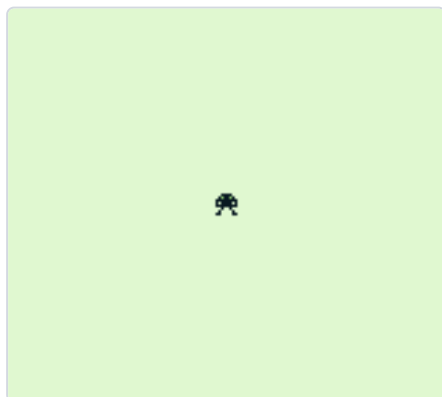
```
int main() {                // Le jeu commence ici.  
    uint8_t x = 76;         // La colonne en pixels (0 à gauche).  
    uint8_t y = 68;         // La ligne en pixels (0 en haut).  
  
    while (true) {          // Répète pour toujours :  
        if (bouton(GAUCHE)) x--; // GAUCHE : x diminue de 1 (« -- » retire 1).  
        if (bouton(DROITE)) x++; // DROITE : x augmente de 1 (« ++ » ajoute 1).  
        if (bouton(HAUT)) y--;   // HAUT : y diminue (on monte : 0 est en haut).  
        if (bouton(BAS)) y++;    // BAS : y augmente (on descend).  
  
        sprite(0, x, y, HEROS); // On repose le lutin 0 à (x, y).  
        image();                // Attend l'image suivante.  
    }                          // Rien ne l'arrête au bord : il peut sortir de l'écran.  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



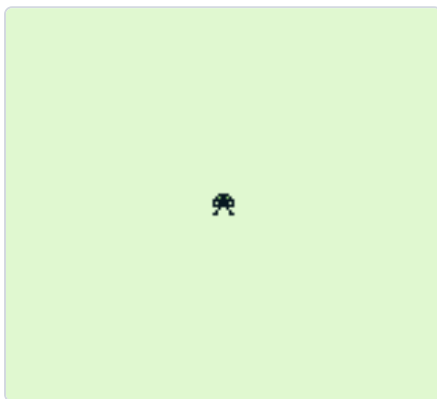
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

La croix déplace le personnage, pixel par pixel.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ DROITE le fait avancer (76 → 96)
- ✓ il avance d'un pixel par image, pas d'une case
- ✓ BAS le fait descendre (88)
- ✓ tenir la touche continue de le déplacer

Ensuite : **49. Le garder dans l'écran** — chapitre 7 : Le mouvement. · [gameboy3](#) — écrire une cartouche Game Boy en C++